スカウトサービス

要件定義 & 技術スタック

ゴール：サービスの全体像と技術構成を理解する

今日の流れ

1. プロジェクト背景
2. ビジョンステートメント
3. コアコンセプト
4. MVP要件（学生側）
5. MVP要件（企業側）
6. MVP要件（システム基盤）
7. 技術スタック
8. プロダクト原則
9. 環境構成 & デプロイフロー & Claude Code レビュー
10. プロジェクト管理
11. ディレクトリ構成

1. プロジェクト背景

現状

面接練習AI
練習内容・回答傾向・スキルレベル

スマートES
ES（自己PR、ガクチカ等）

企業分析AI
調べた企業・業界情報・志望動機

すごい就活
プロフィール情報・就活状況

月間新規登録者 約6,000人
就活生の行動データが各プロダクトに蓄積されている

仮説

月6,000人の登録者がいる

↓

就活生データを企業側に提供

↓

スカウトサービスとしてビジネスが成立するのでは？

2. ビジョン

誰の・どんな課題を・どう解決するか

ビジョンステートメント

使うだけで就活が終わるAIと、
行動データが生む**本質的なマッチング**で、
学生と企業の出会いを変える

ターゲット

誰の・どんな課題を・どう解決するか

toC（学生）

誰の
就活めんどくさいけど楽したい学生

課題
書類作成・ES記入がめんどくさい

解決策
AIが全部作成 → 使うだけでスカウトが届く

toB（企業）

誰の
新卒採用をしている企業

課題
マッチング質低・内定辞退率高・運用工数多

解決策
行動ベースのマッチング + 一括配信で工数削減

3. コアコンセプト

データ統合 → マッチング

データフロー

学生が既存プロダクトを使う

↓

データが自然に溜まる

↓

スカウトサービスでデータ統合

↓

LLM（AI）が学生の本質を可視化

↓

企業がスカウト送信

↓

行動データに基づくマッチング → 学生にスカウトが届く

MVPではこの一連の体験を最小限で成立させる

統合対象データ

データソース	統合するデータ
スマートES	生成したES（自己PR、ガクチカ等）
企業分析AI	調べた企業・業界情報、志望動機
面接練習AI	練習内容、回答傾向、スキルレベル
すごい就活	プロフィール情報、就活状況

分散データを一元化 → 企業が学生を検索・スカウトできるプラットフォーム

4. MVP要件（学生側）

MVP: 学生

MVP Must: 学生側の機能

ログイン
LINE連携 or マジックリンク

データ連携画面
各プロダクトとの連携ON/OFF・同意取得

プロフィール作成
個人情報・各種設定の入力

プロフィール確認
基本情報 + AI統合プロフィール閲覧

公開ON/OFF
企業に見つけてもらうスイッチ

スカウト受信・閲覧
受信一覧・詳細表示・既読管理

スカウト承諾/辞退
ステータス変更

チャット
承諾後に企業とメッセージ

企業視点プレビュー
「企業からこう見えます」の確認

イベント閲覧・申込
説明会・セミナー等への参加

5. MVP要件（企業側）

MVP: 企業 1/2

MVP Must: 企業側の機能（1/2）

アカウント登録・ログイン
マジックリンク認証、owner/adminはMFA必須

企業プロフィール編集
社名・業界・所在地・紹介文・ロゴ等

求人管理
求人の作成・編集・公開/非公開切替

審査フロー
運営審査完了まで学生データ閲覧不可

AIマッチング
採用条件をもとにAIがマッチ学生リスト生成

学生プロフィール閲覧
AI統合プロフィール + 各プロダクトデータ

MVP: 企業 2/2

MVP Must: 企業側の機能（2/2）

スカウト一括配信
マッチした学生に一括送信・重複防止

送信プレビュー
送信前の内容確認

スカウト管理
送信済一覧・ステータス確認

チャット
承諾済学生とメッセージ

ダッシュボード
送信数・承諾率・AIおすすめ学生

イベント管理
説明会等の作成・申込学生管理

メンバー招待
ownerが担当者を追加招待

6. MVP要件（システム基盤）

MVP: システム基盤

MVP Must: システム基盤

4プロダクトからのデータ同期
ETL方式でsynced_*テーブルに格納

AI統合プロフィール生成
LLM（AI）で4プロダクトデータを分析

RLS + 認証基盤
学生/企業の認証・テーブルごとのアクセス制御

通知基盤
LINE通知を基本チャネル + アプリ内通知

流入経路トラッキング
UTM/referrerをサーバー側DBに保存

仮のスカウト送信上限
無課金で月30通の上限設定

7. 技術スタック

Next.js + Supabase + Claude API

技術スタック全体像

フロントエンド
- Next.js (App Router)
- React + TypeScript
- Tailwind CSS
- Shadcn/ui + Radix UI

バックエンド
- Supabase (PostgreSQL)
- Server Actions
- Route Handlers
- Row Level Security

AI・データ処理
- AI統合プロフィール生成
- AIマッチング

認証
- Supabase Auth
- @supabase/ssr
- LINE連携
- MFA (TOTP)

インフラ
- Vercel
- GitHub CI/CD
- Dependabot

モニタリング
- Sentry
- Axiom
- Vercel Analytics

フロントエンド

Next.js (App Router)
Server Componentsでの高速描画、Server Actions / Route HandlersによるAPI層。既存4プロダクトと統一

TypeScript
型安全性の確保。チーム開発でのバグ抑制・コード品質向上

Tailwind CSS
ユーティリティファーストで高速UI開発

Shadcn/ui + Radix UI
UIコンポーネント基盤。ダッシュボード・フォーム・テーブル・モーダル等

バックエンド・認証

Supabase

- PostgreSQL + RLS
- Storage（プロフィール画像等）
- Realtime（チャット）
- 既存4プロダクトと統一

認証方式

学生
LINE連携（推奨）or マジックリンク

企業 owner/admin
マジックリンク + MFA（TOTP必須）

企業 member
マジックリンク（MFA任意）

AI・データ処理 & その他

Claude API

- **AI統合プロフィール生成**: 4プロダクトのデータ分析 → summary, strengths, skills生成
- **AIマッチング**: 企業の採用条件と学生データの照合

Resend

メール送信（スカウト通知・審査完了通知）

Stripe（MVP後）

決済基盤。プラン管理・スカウト上限制御

テスト

Vitest（ユニット）+ Playwright（E2E）

インフラ・モニタリング

インフラ・デプロイ

Vercel
ホスティング・CDN・自動プレビュー環境・WAF

GitHub
ソース管理・CI/CD・PRレビュー・Dependabot

モニタリング

Sentry
エラー監視・パフォーマンス計測

Axiom
ログ集約・分析・ダッシュボード

Vercel Analytics
アクセス解析・Web Vitals

Supabase Logs
Auth・DBクエリログ監視

8. プロダクト原則

プロダクト原則

迷ったときに何を優先するかの判断基準

~~高度な機能~~ → **手軽さ**

~~多くの選択肢~~ → **シンプルで迷いのない選択肢**

やらないこと：toB の学生検索機能

9. 環境構成

Production / Staging / ローカル

環境構成

Production

ブランチ
`main`

Supabase
本番プロジェクト

本番環境。mainにマージで自動デプロイ

Staging

ブランチ
`staging`

Supabase
stagingプロジェクト

本番前の動作確認。stagingにマージで自動デプロイ

ローカル

ブランチ
`feat/* 等`

Supabase
ローカル（Docker）

各開発者の開発・確認用

デプロイフロー & 役割分担

ローカル開発
`feat/xxx` ブランチで開発
各メンバー

↓

Claude Code セルフレビュー
`/review` → 指摘修正 → `/simplify` → 品質向上
各メンバー

↓

staging へ PR
福田がレビュー → マージ → 自動デプロイ
福田がレビュー

↓

main へ PR（本番リリース）
staging → main への昇格 → 自動デプロイ
福田が実施

main / staging への直 push は禁止。必ず PR 経由

Claude Code によるセルフレビュー

PR を出す前に、必ず Claude Code でセルフチェックを行う

/review

目的: 問題の発見・指摘

出力: レビューコメント（指摘のみ）

観点: コードの正しさ、セキュリティ、パフォーマンス

/simplify

目的: 問題の発見 + 自動修正

出力: 修正済みコード + サマリー

観点: コードの再利用性、品質、効率性

PR 前の手順

1. /review を実行 → 2. 指摘を修正 → 3. /simplify で品質向上 → 4. PR 作成

10. プロジェクト管理

GitHub Projects

プロジェクト管理

GitHub Projects

タスク管理・進捗管理を一元化

Issue = タスク

機能単位でIssueを作成。1 Issue = 1 PR のルール

Board ビュー

カンバン形式で進捗を可視化（Todo → In Progress → Done）

コードと直結

PR・ブランチ・レビューがIssueに紐づく。開発の流れがすべて追える

コード管理もタスク管理もGitHub に集約 → ツールの行き来をなくす

11. ディレクトリ構成

App Router とは？

Next.js のルーティング方式。フォルダ構造 = URL構造

フォルダ = ルート

```
src/app/
  page.tsx          → /
  scout/
    page.tsx        → /scout
    [id]/
      page.tsx      → /scout/123
  company/
    dashboard/
      page.tsx      → /company/dashboard
```

page.tsx
そのURLで表示されるページ本体

layout.tsx
共通レイアウト（ヘッダー、サイドバー等）。子ページに自動適用

[id]/（動的ルート）
URLパラメータを受け取るページ。/scout/123 の「123」部分

api/（Route Handlers）
APIエンドポイント。外部サービスとの連携等

ディレクトリ構成

```
scout-product/
  src/
    app/          # Next.js App Router
      api/        # API Routes
      layout.tsx  # ルートレイアウト
      page.tsx    # ホームページ
    components/   # 共通UIコンポーネント
    features/     # 機能モジュール
    lib/          # ユーティリティ
    shared/       # 共有コード
  supabase/
    migrations/   # DBマイグレーション
  e2e/            # E2Eテスト
  docs/           # ドキュメント
```

src/app/
Next.js App Router。ルーティングとページ定義

src/components/
共通UIコンポーネント（ボタン、モーダル等）

src/features/
機能ごとのモジュール（スカウト、チャット等）

src/lib/
Supabaseクライアント、ユーティリティ関数

supabase/
マイグレーションSQL、DB設定

e2e/
Playwright E2Eテスト

Thank you

質問・ディスカッションの時間